

Laboratorio 1

Ottavia M. Epifania
University of Trento
`ottavia.epifania@unitn.it`

A.A. 2025/2026

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo
- 7 Esercizi liberi

Figlio degli anni 90, erede del software **S**

Si tratta di un linguaggio di programmazione object oriented completamente **open source**, gratuito e specificatamente votato alle analisi statistiche

Esiste una comunità mondiale che usa il software e che lavora costantemente per migliorarlo

Fornisce conoscenze trasversali sui linguaggi di programmazione (aiuta ad entrare nelle logiche di programmazione in maniera molto soft)

- 1 Perché
- 2 Installazione**
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo
- 7 Esercizi liberi

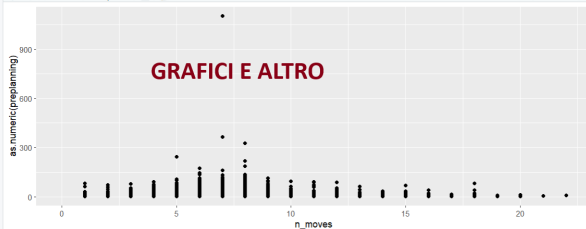
È necessario installare R (disponibile
<https://cran.r-project.org/bin/windows/base/>)

RStudio è l'IDE perfetto per R → consente un utilizzo di R migliore e più semplice

RStudio si può installare dopo aver installato R (disponibile
<https://posit.co/download/rstudio-desktop/>)



AMBIENTE



CONSOLE/SCRIVANIA

Ambiente/Environment. Contiene tutti gli oggetti che sono stati creati fino a quel momento nell'ambiente di lavoro

Script. File di tipo testuale dove salvare il codice usato per generare gli oggetti presenti nell'ambiente. Si possono combinare codice e commenti (righe che non vengono eseguite e interpretate come codice #)

```
# codice per assegnare il numero 25 alla variabile x  
x <- 25
```

Working directory. La cartella dentro cui sta lavorando R in quel momento. Diventa di fondamentale importanza perché è dove R si aspetta che ci siano i file (e.g., i dati) con cui volete lavorare

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ◻ ↺ 🔍 ↻

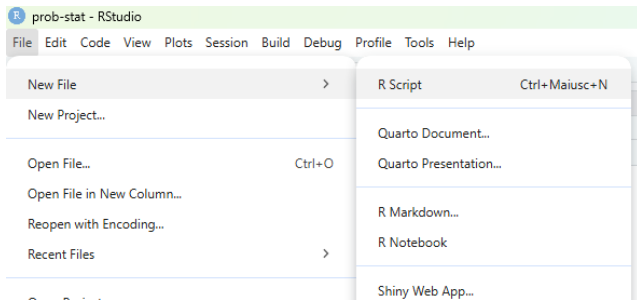


Figure 1: creare un nuovo script

Oppure la sequenza di tasti: `Ctrl + shift + n`

Per salvare uno script: Click sull'icona del salvataggio

- 1 Perché
- 2 **Installazione**
 - Working directories (wd)
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo
- 7 Esercizi liberi

```
# per sapere dove sta lavorando R  
getwd()
```

```
[1] "C:/Users/Ottavia/Documents/GitHub/prob-stat/slides"
```

La wd si può cambiare con il comando `setwd()`:

```
setwd("C:/Users/Ottavia/Documents/altro-percorso")
```

- 1 Perché
- 2 Installazione
- 3 Elementi base**
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo
- 7 Esercizi liberi

- 1 Perché
- 2 Installazione
- 3 Elementi base
 - Calcolatrice
 - Assign
 - Nominare gli oggetti
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo

Ogni parentesi ha un suo significato speciale! Lo vedremo via via

- 1 Perché
- 2 Installazione
- 3 Elementi base
 - Calcolatrice
 - Assign
 - Nominare gli oggetti
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo

Tutto quello che viene creato in R viene chiamato *oggetto* tramite l' "assegnazione" con specifici simboli, ovvero `<-` oppure `=`:

```
x <- 4 ; x
```

```
[1] 4
```

```
X = 5 ; X
```

```
[1] 5
```

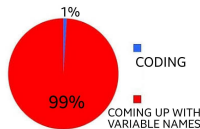
Gli elementi a destra del simbolo (contenuto) vengono assegnati all'elemento a sinistra (contenitore) del simbolo

Attenzione!

R è case sensitive. `x` e `X` sono due oggetti diversi!

- 1 Perché
- 2 Installazione
- 3 Elementi base
 - Calcolatrice
 - Assign
 - Nominare gli oggetti
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo

Nominare gli oggetti



Nomi validi sono lettere, numeri, punti, underscore (e.g., `variable_name`)

I nomi NON possono iniziare con numeri, simboli o valori interni di R

Buoni nomi per le variabili:

- `mean_rt`
- `sum_age`
- `prop_item`

Pessimi nomi per variabili:

- `x`
- `y`
- `ciccio`

Nominare gli oggetti

```
.01 = 5
```

Error in 0.01 = 5: membro di sinistra dell'assegnazione (do_set) non valido

```
1 = "a"
```

Error in 1 = "a": membro di sinistra dell'assegnazione (do_set) non valido

```
TRUE = 567
```

Error in TRUE = 567: membro di sinistra dell'assegnazione (do_set) non valido



- Creare una cartella sul vostro Desktop/Scrivania nominata `corso-statistica`
- Aprire RStudio e creare un nuovo script
- Svolgere le sgeuenti operazioni. Il risultato di ogni operazione deve essere assegnato a un oggetto:
 - $10 + 15$ (nome dell'oggetto: `somma`)
 - $\frac{5}{10} \times 3 - 2$ (nome dell'oggetto: `operazione_1`)
 - $(3 * 2) + \frac{5}{10} \times (45 * 0.5) - 15$ (nome dell'oggetto: `operazione_2`)

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture**
- 5 Funzioni utili
- 6 Monte Carlo
- 7 Esercizi liberi

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture**
 - Funzioni
 - Vettori
 - Indicizzare i vettori
 - Matrici
 - Indicizzare le matrici: In pratica
- 5 Funzioni utili

Sono i comandi che permettono di fare tutto.

Ne esistono di predefinite all'installazione di R, se ne possono definire di nuove autonomamente, si possono installare dei “pacchetti” dal CRAN (comprehensive R archive network) o da un file locale, a seconda delle operazioni che si vogliono svolgere.

Ad esempio, per installare il pacchetto `ggplot2` (per dei grafici meravigliosi) non si deve far altro che scrivere in console:

```
install.packages("ggplot2")  
# per rendere il pacchetto operativo  
library(ggplot2) # permette di usare le funzioni al suo interno
```

L'installazione da file locale presuppone che si abbia il file sorgente del pacchetto a disposizione.

I file sorgenti dei pacchetti hanno estensione `tar.gz`.

Per installare il pacchetto `rmf` (necessario per il corso e scaricabile dal moodle del corso):

```
install.packages("rmf_3.0-03000.tar.gz", repos = NULL, type = "source")  
# per rendere usabili le funzioni  
library(rmf)
```

```
mean(x, trim = 0, na.rm = FALSE, ...)
```

- Nome della funzione: `mean`
- Parentesi che racchiudono gli argomenti della funzione
- argomenti:
 - obbligatori, senza valori predefiniti senza di cui la funzione non può funzionare
 - facoltativi con valori di default (`trim = 0`, `na.rm = FALSE`)

```
mean(1:5)
```

```
[1] 3
```

```
mean(1:5, trim = 0.3)
```

```
[1] 3
```

Alcune funzioni non richiedono argomenti al loro interno:

```
ls() # cosa c'è nell'ambiente di R
```

```
[1] "myinclude" "x"          "X"
```

```
dir() # cosa c'è nella wd
```

```
[1] "img"          "Lab01.pdf"      "Lab01.qmd"      "Lab01.rmarkc
[5] "Lab02.log"     "Lab02.qmd"      "Lab02.tex"      "Lab02_files"
```

?funzione permette di accedere all'help della funzione:

?mean

Environment Files Plots Packages Help Viewer Presentation

R: Arithmetic Mean Find in Topic

mean (base) R Documentation

Arithmetic Mean

Description

Generic function for the (trimmed) arithmetic mean.

Usage

```
mean(x, ...)
```

Default S3 method:

```
mean(x, trim = 0, na.rm = FALSE, ...)
```

Arguments

- x** an R object. Currently there are methods for numeric/logical vectors and [date](#), [date-time](#) and [time interval](#) objects. Complex vectors are allowed for `trim = 0`, only.
- trim** the fraction (0 to 0.5) of observations to be trimmed from each end of `x` before the mean is computed. Values of trim outside that range are taken as the nearest endpoint.
- na.rm** a logical evaluating to TRUE or FALSE indicating whether NA values should be stripped before the computation proceeds.
- ...** further arguments passed to or from other methods.

concatenate, c()

Concatena diversi oggetti insieme per creare un nuovo oggetto unico

```
x = c(1, 2, 3) # crea un vettore concatenando 1,2,3  
x
```

```
[1] 1 2 3
```

```
X = 1:3 # crea lo stesso identico vettore  
X
```

```
[1] 1 2 3
```

```
x == X
```

```
[1] TRUE TRUE TRUE
```

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture**
 - Funzioni
 - Vettori
 - Indicizzare i vettori
 - Matrici
 - Indicizzare le matrici: In pratica
- 5 Funzioni utili

Tipi di oggetti diversi → Tipi di vettori diversi:

- `num` (`numeric`): Numeri (-0.56, 3, 4.5)
- `logi` (`logic`): Logico (TRUE vs FALSE)
- `chr` (`character`): characters ("a", "b", "c")
- `factor` (`factor`): factor con diversi livelli

La distinzione più importanti è tra vettori di tipo categoriale (`chr` e `factor`) e vettori di tipo numerico (`int` e `num`):

- `integer` e `numeric`: Operazioni matematiche E calcolo delle frequenze
- `character` e `factor`: Calcolo delle frequenze

```
months = c(5, 6, 8, 10, 12, 16); months
```

```
[1]  5  6  8 10 12 16
```

```
weight = seq(3, 11, by = 1.5) ; weight
```

```
[1] 3.0 4.5 6.0 7.5 9.0 10.5
```

I vettori logici si ottengono testando delle condizioni sulla base dei valori di altri vettori:

```
months
```

```
[1]  5  6  8 10 12 16
```

Prendere solo i valori > 12

```
months > 12
```

```
[1] FALSE FALSE FALSE FALSE FALSE  TRUE
```

```
v_chr = c(letters[1:3], LETTERS[4:6]); v_chr
```

```
[1] "a" "b" "c" "D" "E" "F"
```

```
ses = factor(rep(c("low", "medium", "high"), each = 2)); ses
```

```
[1] low    low    medium medium high    high  
Levels: high low medium
```

Attenzione a non mischiare gli oggetti!!

`num + logi → num`

`num + factor → num`

`num + chr → chr`

`chr + logi → chr`

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture**
 - Funzioni
 - Vettori
 - Indicizzare i vettori
 - Matrici
 - Indicizzare le matrici: In pratica
- 5 Funzioni utili

```
nomi = c("Pasquale", "Egidio", "Debora", "Luca", "Andrea")
nomi
```

```
[1] "Pasquale" "Egidio" "Debora" "Luca" "Andrea"
```

Pasquale	Egidio	Debora	Luca	Andrea
----------	--------	--------	------	--------

1

2

3

4

5

```
nome_vettore[indice]
```


Pasquale	Egidio	Debora	Luca	Andrea
1	2	3	4	5

nomi[1] $\rightarrow$

Pasquale	Egidio	Debora	Luca	Andrea
1	2	3	4	5

`nomi[1]` → Pasquale

`nomi[3]` →

Pasquale	Egidio	Debora	Luca	Andrea
1	2	3	4	5

nomi[1] → Pasquale

nomi[3] $\rightarrow$ Debora

```
nomi[seq(2, 5, by = 2)] →
```

Pasquale	Egidio	Debora	Luca	Andrea
1	2	3	4	5

nomi[1] → Pasquale

nomi[3] $\rightarrow$ Debora

```
nomi[seq(2, 5, by = 2)] → Egidio, Luca
```

Indicizzare i vettori

weight

```
[1] 3.0 4.5 6.0 7.5 9.0 10.5
```

```
weight[2] # secondo elemento di weight
```

[1] 4.5

```
(weight[6] = 15.2) # sostituisce il sesto elemento di weight
```

[1] 15.2

```
weight[seq(1, 6, by = 2)] # elementi 1, 3, 5
```

[1] 3 6 9

```
weight[2:6] # elementi dal 2 al 6 (incluso)
```

```
[1] 4.5 6.0 7.5 9.0 15.2
```

Si può indicizzare con la logica

```
weight > 7
```

```
[1] FALSE FALSE FALSE  TRUE  TRUE  TRUE
```

```
weight[weight > 7] # solo i valori > di 7
```

```
[1]  7.5  9.0 15.2
```

```
weight[weight >= 4.5 & weight < 8] # valori compresi tra 4.5 e 8
```

```
[1] 4.5 6.0 7.5
```



- Assegna i seguenti valori a un oggetto chiamato `my_var` (i valori devono essere **concatenati** tra loro)
23, 24, 25, 27, 28, 29, 30
- Calcola la media di `my_var` usando la funzione `mean()`, il minimo (`min()`) e massimo (`max()`)
- Crea un oggetto (`voto`) di tipo `character` come segue (usare la funzione `rep()`):
 - “si” ripetuto 12 volte
 - “no” ripetuto 15 volte

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture**
 - Funzioni
 - Vettori
 - Indicizzare i vettori
 - Matrici
 - Indicizzare le matrici: In pratica
- 5 Funzioni utili

```
# crea una matrice con 3 righe e 4 colonne
# (i valori sono sistemati per riga)
A = matrix(1:12, nrow=3, ncol = 4, byrow = TRUE)
# assegna dei nomi alle righe
rownames(A) = paste("a",
                     1:nrow(A), sep = "_")
# assegna dei nomi alle colonne
colnames(A) = paste("b",
                    1:ncol(A), sep = "_")
```

A

	b_1	b_2	b_3	b_4
a_1	1	2	3	4
a_2	5	6	7	8
a_3	9	10	11	12

	[,1]	[,2]	[,3]
[1,]	1, 1	1, 2	1, 3
[2,]	2, 1	2, 2	2, 3
[3,]	3, 1	3, 2	3, 3

```
my_matrix[righe, colonne]
```

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture**
 - Funzioni
 - Vettori
 - Indicizzare i vettori
 - Matrici
 - Indicizzare le matrici: In pratica
- 5 Funzioni utili

A

	b_1	b_2	b_3	b_4
a_1	1	2	3	4
a_2	5	6	7	8
a_3	9	10	11	12

$$A[1, \] \rightarrow 1, 2, 3, 4$$
$$A[2,] \rightarrow 5, 6, 7, 8$$
$$A[2, 3] \rightarrow 7$$



- Crea una matrice 3×3 con la tabellina del 3 fino a 24, assegnandola all'oggetto `my_mat`
- Dai un nome alle righe (e.g., `r1`, `r2`, `r3`) e alle colonne (e.g., `c1`, `c2`, `c3`)
- Indice da `my_mat`:
 - La prima riga
 - La seconda colonna
 - La cella `[1, 2]`

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili**
- 6 Monte Carlo
- 7 Esercizi liberi

- `mean()`: calcola la media di un vettore numerico
- `sd()`: calcola la deviazione standard di un vettore numerico
- `table()`: calcola le frequenze per i livelli di una variabile
- `sort()`: ordina una variabile in base ai valori (dal più piccolo al più grande)
- `sample()`: permuta i valori all'interno di un vettore, permette di creare vettori casuali di valori ecc

```
my_var = c(23, 24, 27, 28, 29, 30); my_var
```

```
[1] 23 24 27 28 29 30
```

```
voto = c(rep("si", 12), rep("no", 15)); voto
```

```
[1] "si" "si" "si" "si" "si" "si" "si" "si" "si" "si" "si" "si" "no" "no" "no"  
[16] "no" "no" "no" "no" "no" "no" "no" "no" "no" "no" "no" "no"
```

```
table(my_var)
```

```
my_var
```

23	24	27	28	29	30
1	1	1	1	1	1

```
table(voto)
```

voto

no si

15 12

```
sample(x, size, replace = FALSE, prob = NULL)
```

```
set.seed(2026) # ci assicura di osservare sempre lo stesso risultato  
voto
```

```
[1] "si" "si" "si" "si" "si" "si" "si" "si" "si" "si" "si" "si" "no" '  
[16] "no" "no" "no" "no" "no" "no" "no" "no" "no" "no" "no" "no"
```

```
sample(voto)
```

```
[1] "no" "si" "si" "no" "no" "si" "si" "no" "si" "no" "si" "si" "si" '  
[16] "no" "si" "no" "no" "si" "si" "no" "no" "no" "no" "si" "no"
```

Estrarre un campione di ampiezza 10 dal vettore

```
sample(voto, size = 10)
```

```
[1] "si" "si" "no" "si" "si" "si" "no" "si" "no" "si"
```

Creare un nuovo vettore di lunghezza 10 dove il sì abbia una probabilità del 70% e il no una probabilità del 30%

```
new_voto = sample(c("si", "no"), size = 10, replace = TRUE,  
                  prob = c(.70, .30))  
new_voto
```

```
[1] "si" "no" "si" "no" "no" "si" "si" "no" "si" "si"
```

```
table(new_voto)/length(new_voto)
```

```
new_voto  
  no  si  
0.4 0.6
```

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili**
 - Basi di programmazione: Il ciclo for
- 6 Monte Carlo
- 7 Esercizi liberi

Serve per iterare un numero predefinito di volte la stessa operazione (loop)

```
for (variable in vector) {  
  computation  
}
```

variable: un indice che si muove attraverso **vector**

vector: definisce il numero di iterazioni

computation: operazioni che vengono svolte all'interno del ciclo **for**

```
replicate = 100
results = numeric(100)
set.seed(123)
for (i in 1:replicate) {
  temp_variable = rexp(200, rate = i)
  results[i] = mean(temp_variable)
}
length(results)
```

①
②
③
④
⑤

- ① Definisce il numero di iterazioni
- ② Inizializza un vettore di lunghezza 100 per salvare i risultati
- ③ “Pianta il seme” per rendere i risultati replicabili
- ④ La variabile *i* si muove attraverso i valori che vanno da 1 a 100 (1:replicate)
- ⑤ Qui si svolge realmente l'operazione di generare i numeri casuali da una distribuzione esponenziale dove il *rate* che ne definisce la velocità di accelerazione è definito da *i* mentre alla riga successiva si calcola la media per ogni variabile e la si assegna all'*i*-esima posizione del vettore *results*

[1] 100

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo**
- 7 Esercizi liberi

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo**
 - Esempio: La moneta è truccata?
- 7 Esercizi liberi

Esempio: La moneta è truccata?

1. Lo scenario

```
moneta = c("testa", "croce")
```

2. Numero di repliche

```
repliche = 1000
```

3. Ripetere lo scenario & salvare i risultati

```
mc = sample(moneta, repliche, replace = TRUE)
# lunghezza del vettore
length(mc) == repliche
```

```
[1] TRUE
```

```
str(mc)
```

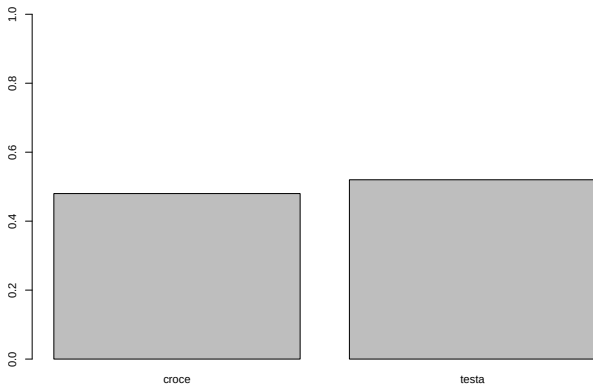
```
chr [1:1000] "croce" "testa" "testa" "croce" "testa" "testa" "
```

◀ ◻ ▶ ◀ ◻ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ↺ 🔍 ↻

Esempio: La moneta è truccata?

4. Calcolare le statistiche di interesse

```
barplot(prop.table(table(mc)), ylim=c(0,1))
```



Esempio: La moneta è truccata?

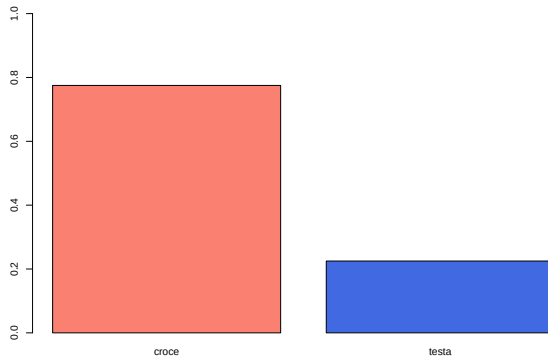


Figure 2: Questa moneta è truccata? Provate a ricostruire il codice!

P.S. :set.seed(1903)

Soluzione

```
newmoneta = sample(moneta, repliche,  
                    replace = TRUE, prob = c(.20, .80))  
barplot(prop.table(table(newmoneta)),  
        ylim= c(0,1), col = c("salmon", "royalblue"))
```

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo
- 7 Esercizi liberi**

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo
- 7 Esercizi liberi**
 - Esempio 1.5 (pag. 24)
 - Esempio 1.6

Considerato un mazzo di carte ($n = 52$), qual è la probabilità di estrarre una carta di fiori?

La soluzione analitica è molto semplice: Ci sono 13 carte fiori, quindi la probabilità di estrarre una carta fiori è di $\frac{13}{52} = 0.25$.

Trovare lo stesso risultato con una simulazione di Monte Carlo

Codice per creare il mazzo

```
mazzo = c("fiori", "cuori", "picche", "denari")  
mazzo = paste0(mazzo, rep(1:13, each = 4))
```

Soluzione

```
mazzo = c("fiori", "cuori", "picche", "denari")
mazzo = paste0(mazzo, rep(1:13, each = 4))
set.seed(123)
repliche = 1000
results = numeric(repliche)
for (i in 1:repliche) {
  results[i] = sample(mazzo, 1)
}
table(results)
sum(grepl("fiori", results))/repliche
```

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo
- 7 Esercizi liberi**
 - Esempio 1.5 (pag. 24)
 - Esempio 1.6

Considerato un mazzo di carte ($n = 52$), qual è la probabilità di estrarre un asso?

La soluzione analitica è molto semplice: Ci sono 4 assi (uno per ogni seme), per cui la probabilità di trovare un asso è $\frac{4}{52} = 0.0769231$.

Trovare lo stesso risultato con una simulazione di Monte Carlo

Codice per creare il mazzo

```
mazzo = c("fiori", "cuori", "picche", "denari")  
mazzo = paste(mazzo, rep(c("asso", 2:13), each = 4), sep = "_")
```

Soluzione

```
mazzo = c("fiori", "cuori", "picche", "denari")
mazzo = paste(mazzo, rep(c("asso", 2:13), each = 4), sep = "_")
set.seed(123)
repliche = 1000
results = numeric(repliche)
for (i in 1:repliche) {
  results[i] = sample(mazzo, 1)
}
table(results)
sum(grepl("asso", results))/repliche
```

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo
- 7 Esercizi liberi**
 - Esempio 1.5 (pag. 24)
 - Esempio 1.6

52 carte, ne vengono estratte due. Calcolare la probabilità che entrambe siano fiori, considerando due scenari:

- con reimmissione (eventi indipendenti)
- senza reimmissione (eventi dipendenti)

Con reimmissione

$A =$ Entrambe le carte sono fiori

Casi possibili: 52^2

Casi favorevoli: 13^2

$$P(A) = \frac{13^2}{52^2} = \frac{1}{16} = 0.0625$$

Senza reimmissione

$A =$ Entrambe le carte sono fiori

Casi possibili: $52^2 - 52$

Casi favorevoli: $13^2 - 13$

$$P(A) = \frac{13^2 - 13}{52^2 - 52} = \frac{3}{51} = 0.05882353$$

MC?

Soluzione (con reinserimento)

```
mazzo = rep(c("cuori", "denari", "fiori", "picche"), each = 13)
repliche = 10000
results1 = 10000
for (i in 1:repliche) {
  out = sample(mazzo, size = 2, replace = TRUE)
  results1[i] = all(out == "fiori")
}
sum(results1) # sono casi favorevoli
```

```
[1] 597
```

```
sum(results1)/repliche
```

```
[1] 0.0597
```

Soluzione (senza reinserimento)

```
mazzo = rep(c("cuori", "denari", "fiori", "picche"), each = 13)
repliche = 10000
results2 = 10000
for (i in 1:repliche) {
  out = sample(mazzo, size = 2, replace = FALSE)
  results2[i] = all(out == "fiori")
}
sum(results2) # sono casi favorevoli
```

```
[1] 581
```

```
sum(results2)/repliche
```

```
[1] 0.0581
```

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo
- 7 Esercizi liberi**
 - Esempio 1.5 (pag. 24)
 - Esempio 1.6

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

Soluzione

```
moneta = c("T", "C")
repliche = 10000
results = numeric(repliche)

for (i in 1:repliche) {
  out = sample(moneta, 10, replace = TRUE)
  results[i] = sum(out == "C") >= 6
}

sum(results)
```

[1] 3867

```
sum(results)/repliche
```

```
[1] 0.3867
```

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo
- 7 Esercizi liberi**
 - Esempio 1.5 (pag. 24)
 - Esempio 1.6

Si hanno due urne definite come segue:

Urna 1

Una pallina verde (V)

Una pallina blu (B)

Urna 2

Una pallina verde (V)

Una pallina rossa (R)

Calcolare la probabilità che estraenedo una pallina da ogni urna, le due palline siano dello stesso colore

Si hanno due urne definite come segue:

Urna 1

Una pallina verde (V)

Una pallina blu (B)

Urna 2

Una pallina verde (V)

Una pallina rossa (R)

Calcolare la probabilità che estraenedo una pallina da ogni urna, le due palline siano dello stesso colore

$$\Omega = \{(V, V), (V, R), (B, V), (B, R)\}$$

$$|\Omega| = 4$$

Unico caso favorevole: $A = \{(V, V)\}$ ($|A| = 1$)

Ne segue che:

$$P(A) = \frac{|A|}{|\Omega|} = \frac{1}{4} = 0.25$$

MC? (`set.seed(123)`)

Soluzione

```
set.seed(123)
u1 = c("verde", "blu")
u2 = c("verde", "rossa")
repliche = 1000
results = numeric(repliche)
for (i in 1:repliche) {
  out1 = sample(u1,1)
  out2 = sample(u2, 1)
  results[i] = all(out1 == "verde", out2 == "verde")
}
table(results)/repliche
```

```
results
      0      1
0.736 0.264
```

- 1 Perché
- 2 Installazione
- 3 Elementi base
- 4 Strutture
- 5 Funzioni utili
- 6 Monte Carlo
- 7 Esercizi liberi**
 - Esempio 1.5 (pag. 24)
 - Esempio 1.6

4 palline rosse, 3 palline blu

Vengono fatte due estrazioni *senza reinserimento*

A = La prima pallina estratta è rossa

B = La seconda pallina estratta è rossa

Codice per generare l'urna

```
urna = c(rep("rossa", 4), rep("blu", 3))
```

Soluzione

```
repliche = 1000
resA = numeric(1000)
resB = numeric(1000)
den = 0
for (i in 1:repliche) {
  out = sample(urna, size = 2)
  den = den + 1
  resA[i] = out[1] == "rossa"
  resB[i] = out[2] == "rossa"
}
sum(resA) # casi favorevoli ad A (la prima pallina è rossa)
```

[1] 586

```
sum(resB) # casi favorevoli a B (la seconda pallina è rossa)
```

[1] 579